0.1 Take an alphabet (A–Z or a–z) and a digit N (0–9); display the Nth character from that alphabet starting from the input letter (same case)

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG1 DB 'Input Alphabet: $'
    MSG2 DB 0DH, 0AH, 'Given Value N: $'
    MSG3 DB 0DH, 0AH, 'Output: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    MOV BL, AL

    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    SUB AL, 48

    ADD BL, AL

    LEA DX, MSG3
    MOV AH, 9
    INT 21H

    MOV DL, BL
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.2 Input 5 lowercase letters; display them in reverse order in uppercase

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Output: $'
    ARR DB 5 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0

INPUT_LOOP:
    MOV AH, 1
    INT 21H
    SUB AL, 32
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 4

OUTPUT_LOOP:
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    DEC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.3 Input 5 uppercase letters; display them in reverse order in lowercase.

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Output: $'
    ARR DB 5 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0

INPUT_LOOP:
    MOV AH, 1
    INT 21H
    ADD AL, 32
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 4

OUTPUT_LOOP:
```

```
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    DEC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.4 Input a digit N (0–9); display all even digits up to N

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Even Digits: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    SUB AL, 48
    MOV CL, AL

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV BL, 0

PRINT_LOOP:
    CMP BL, CL
    JG EXIT_PROG

    MOV DL, BL
    ADD DL, 48
    MOV AH, 2
    INT 21H

    ADD BL, 2
    JMP PRINT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.5 Input a digit N (0–9); display all odd digits up to N

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Odd Digits: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    SUB AL, 48
    MOV CL, AL

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV BL, 1

PRINT_LOOP:
    CMP BL, CL
    JG EXIT_PROG

    MOV DL, BL
    ADD DL, 48
    MOV AH, 2
    INT 21H

    ADD BL, 2
    JMP PRINT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.6 Input 5 digits (0–9); arrange them in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Ascending: $'
    ARR DB 5 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0

INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 4
```

```
OUTER_LOOP:
    PUSH CX
    MOV SI, 0
    MOV CX, 4
INNER_LOOP:
    MOV AL, ARR[SI]
    CMP AL, ARR[SI+1]
    JL SKIP_SWAP
    XCHG AL, ARR[SI+1]
    MOV ARR[SI], AL
SKIP_SWAP:
    INC SI
    LOOP INNER_LOOP
    POP CX
    LOOP OUTER_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0
OUTPUT_LOOP:
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.7 Input 5 digits (0–9); arrange them in descending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Descending: $'
    ARR DB 5 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0

INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 4
OUTER_LOOP:
    PUSH CX
    MOV SI, 0
    MOV CX, 4
INNER_LOOP:
    MOV AL, ARR[SI]
    CMP AL, ARR[SI+1]
    JG SKIP_SWAP
    XCHG AL, ARR[SI+1]
    MOV ARR[SI], AL
SKIP_SWAP:
    INC SI
    LOOP INNER_LOOP
    POP CX
    LOOP OUTER_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 4
OUTPUT_LOOP:
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    DEC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.8 Input 5 digits (0–9); separate and display odd and even digits

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    MSG_ODD DB 0DH, 0AH, 'Odd: $'
    MSG_EVEN DB 0DH, 0AH, 'Even: $'
    ARR DB 5 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    LEA DX, MSG_ODD
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0
ODD_LOOP:
    MOV AL, ARR[SI]
    TEST AL, 1
```

```
    JZ SKIP_ODD
    MOV DL, AL
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
SKIP_ODD:
    INC SI
    LOOP ODD_LOOP

    LEA DX, MSG_EVEN
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV SI, 0
EVEN_LOOP:
    MOV AL, ARR[SI]
    TEST AL, 1
    JNZ SKIP_EVEN
    MOV DL, AL
    MOV AH, 2
    INT 21H
    MOV DL, ' '
    INT 21H
SKIP_EVEN:
    INC SI
    LOOP EVEN_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.9 Input 8 digits (0–9); sort only the odd digits in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Odd Ascending: $'
    ARR DB 8 DUP(?)
    ODD_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_ODD:
    MOV AL, ARR[SI]
    TEST AL, 1
    JZ SKIP_FILTER
    MOV ODD_ARR[DI], AL
    INC DI
    INC COUNT
SKIP_FILTER:
    INC SI
    LOOP FILTER_ODD

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, ODD_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JL NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, ODD_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.10 Input 8 digits (0–9); sort only the odd digits in descending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Odd Descending: $'
    ARR DB 8 DUP(?)
    ODD_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
```

```
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_ODD:
    MOV AL, ARR[SI]
    TEST AL, 1
    JZ SKIP_FILTER
    MOV ODD_ARR[DI], AL
    INC DI
    INC COUNT
SKIP_FILTER:
    INC SI
    LOOP FILTER_ODD

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, ODD_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JAE NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, ODD_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.11 Input 8 digits (0–9); sort only the even digits in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Even Ascending: $'
    ARR DB 8 DUP(?)
    EVEN_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_EVEN:
    MOV AL, ARR[SI]
    TEST AL, 1
    JNZ SKIP_FILTER
    MOV EVEN_ARR[DI], AL
    INC DI
    INC COUNT
SKIP_FILTER:
    INC SI
    LOOP FILTER_EVEN

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, EVEN_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JL NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
```

```
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, EVEN_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.12 Input 8 digits (0–9); sort only the even digits in descending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Even Descending: $'
    ARR DB 8 DUP(?)
    EVEN_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_EVEN:
    MOV AL, ARR[SI]
    TEST AL, 1
    JNZ SKIP_FILTER
    MOV EVEN_ARR[DI], AL
    INC DI
    INC COUNT
SKIP_FILTER:
    INC SI
    LOOP FILTER_EVEN

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, EVEN_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JAE NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, EVEN_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.13 Input 8 digits (0–9); find prime digits and display them in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Primes Digits Ascending: $
        '
    ARR DB 8 DUP(?)
    PRIME_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
```

```
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_PRIME:
    MOV AL, ARR[SI]

    CMP AL, '2'
    JE IS_PRIME
    CMP AL, '3'
    JE IS_PRIME
    CMP AL, '5'
    JE IS_PRIME
    CMP AL, '7'
    JE IS_PRIME
    JMP SKIP_PRIME

IS_PRIME:
    MOV PRIME_ARR[DI], AL
    INC DI
    INC COUNT

SKIP_PRIME:
    INC SI
    LOOP FILTER_PRIME

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, PRIME_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JL NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, PRIME_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.14 Input 8 digits (0–9); find prime digits and display them in descending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Primes Digits Descending:
        $'
    ARR DB 8 DUP(?)
    PRIME_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_PRIME:
    MOV AL, ARR[SI]
    CMP AL, '2'
    JE IS_PRIME
    CMP AL, '3'
    JE IS_PRIME
    CMP AL, '5'
    JE IS_PRIME
    CMP AL, '7'
    JE IS_PRIME
    JMP SKIP_PRIME

IS_PRIME:
    MOV PRIME_ARR[DI], AL
    INC DI
    INC COUNT

SKIP_PRIME:
    INC SI
    LOOP FILTER_PRIME

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
```

```
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, PRIME_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JAE NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, PRIME_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.15 Input 8 digits (0–9); find odd prime digits and display them in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Odd Primes Ascending: $'
    ARR DB 8 DUP(?)
    OPRIME_ARR DB 8 DUP(?)
    COUNT DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 8
    MOV SI, 0
    MOV DI, 0
FILTER_OPRIME:
    MOV AL, ARR[SI]

    CMP AL, '3'
    JE IS_OPRIME
    CMP AL, '5'
    JE IS_OPRIME
    CMP AL, '7'
    JE IS_OPRIME
    JMP SKIP_FILTER

IS_OPRIME:
    MOV OPRIME_ARR[DI], AL
    INC DI
    INC COUNT

SKIP_FILTER:
    INC SI
    LOOP FILTER_OPRIME

    MOV AL, COUNT
    CMP AL, 1
    JBE DISPLAY_RESULT

    DEC AL
    XOR CH, CH
    MOV CL, AL
OUTER:
    PUSH CX
    LEA SI, OPRIME_ARR
INNER:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JL NEXT_STEP
    XCHG AL, [SI+1]
    MOV [SI], AL
NEXT_STEP:
    INC SI
    LOOP INNER
    POP CX
    LOOP OUTER

DISPLAY_RESULT:
    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    XOR CH, CH
    MOV CL, COUNT
    JCXZ EXIT_PROG
    LEA SI, OPRIME_ARR
OUTPUT_LOOP:
    MOV DL, [SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

EXIT_PROG:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.16 Input 8 digits (0–9); sort all digits in ascending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Output: $'
    ARR DB 8 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0

INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 7
OUTER_LOOP:
    PUSH CX
    MOV SI, 0
    MOV CX, 7
INNER_LOOP:
    MOV AL, ARR[SI]
    CMP AL, ARR[SI+1]
    JL SKIP_SWAP
    XCHG AL, ARR[SI+1]
    MOV ARR[SI], AL
SKIP_SWAP:
    INC SI
    LOOP INNER_LOOP
    POP CX
    LOOP OUTER_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
OUTPUT_LOOP:
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.17 Input 8 digits (0–9); sort all digits in descending order

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Output: $'
    ARR DB 8 DUP(?)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 1
    INT 21H
    MOV ARR[SI], AL
    INC SI
    LOOP INPUT_LOOP

    MOV CX, 7
OUTER_LOOP:
    PUSH CX
    MOV SI, 0
    MOV CX, 7
INNER_LOOP:
    MOV AL, ARR[SI]
    CMP AL, ARR[SI+1]
    JAE SKIP_SWAP
    XCHG AL, ARR[SI+1]
    MOV ARR[SI], AL
SKIP_SWAP:
    INC SI
    LOOP INNER_LOOP
    POP CX
    LOOP OUTER_LOOP

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV CX, 8
    MOV SI, 0
OUTPUT_LOOP:
    MOV DL, ARR[SI]
    MOV AH, 2
    INT 21H
    INC SI
    LOOP OUTPUT_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.18 Input 2 digits (0–9); find their Greatest Common Divisor (GCD)

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'GCD: $'
```

```
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    SUB AL, 48
    MOV BL, AL

    MOV AH, 1
    INT 21H
    SUB AL, 48
    MOV BH, AL

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    CMP BL, 0
    JE SHOW_BH
    CMP BH, 0
    JE SHOW_BL

GCD_LOOP:
    CMP BL, BH
    JE DONE
    JB SWAP_VAL
    SUB BL, BH
    JMP GCD_LOOP

SWAP_VAL:
    XCHG BL, BH
    JMP GCD_LOOP

SHOW_BH:
    MOV BL, BH
    JMP DONE

SHOW_BL:
    JMP DONE

DONE:
    MOV DL, BL
    ADD DL, 48
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.19 Input 5 digits (0–9); find their summation and display the result in hexadecimal

```
.MODEL SMALL
.STACK 100H

.DATA
    PROMPT DB 'Input: $'
    RESULT DB 0DH, 0AH, 'Sum: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 9
    INT 21H

    MOV CX, 5
    MOV BL, 0

INPUT_SUM:
    MOV AH, 1
    INT 21H
    SUB AL, 48
    ADD BL, AL
    LOOP INPUT_SUM

    LEA DX, RESULT
    MOV AH, 9
    INT 21H

    MOV AL, BL
    AND AL, 0F0H
    SHR AL, 4

    CMP AL, 9
    JBE FIRST_DIGIT
    ADD AL, 7
FIRST_DIGIT:
    ADD AL, 48
    MOV DL, AL
    MOV AH, 2
    INT 21H

    MOV AL, BL
    AND AL, 0FH

    CMP AL, 9
    JBE SECOND_DIGIT
    ADD AL, 7
SECOND_DIGIT:
    ADD AL, 48
    MOV DL, AL
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.20 Display the string "Islamic University of Technology" and its reverse alternately 10 times each (total 20 lines).

```
.MODEL SMALL
.STACK 100H

.DATA
    STR DB 'Islamic University of Technology$'
    LEN DW 32
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```

```
    MOV CX, 10

OUTER_LOOP:
    PUSH CX

    LEA DX, STR
    MOV AH, 9
    INT 21H

    LEA DX, NEWLINE
    MOV AH, 9
    INT 21H

    MOV CX, LEN
    MOV SI, LEN
    DEC SI

REV_LOOP:
    MOV DL, STR[SI]
    MOV AH, 2
    INT 21H
    DEC SI
    LOOP REV_LOOP

    LEA DX, NEWLINE
    MOV AH, 9
    INT 21H

    POP CX
    LOOP OUTER_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.21 Display all ASCII characters in reverse order

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG DB 'ASCII Characters in Reverse Order:', 0
        DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG
    MOV AH, 9
    INT 21H

    MOV CX, 256
    MOV DL, 255

REVERSE_ASCII:
    MOV AH, 2
    INT 21H

    DEC DL
    LOOP REVERSE_ASCII

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.22 Display ASCII characters of digits and alphabets (upper and lower case) in forward and reverse order

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG_F DB 'Forward Order:', 0DH, 0AH, '$'
    MSG_R DB 0DH, 0AH, 'Reverse Order:', 0DH, 0AH,
        '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG_F
    MOV AH, 9
    INT 21H

    MOV DL, '0'
L1:
    MOV AH, 2
    INT 21H
    INC DL
    CMP DL, '9'
    JBE L1

    MOV DL, 'A'
L2:
    MOV AH, 2
    INT 21H
    INC DL
    CMP DL, 'Z'
    JBE L2

    MOV DL, 'a'
L3:
    MOV AH, 2
    INT 21H
    INC DL
    CMP DL, 'z'
    JBE L3

    LEA DX, MSG_R
    MOV AH, 9
    INT 21H

    MOV DL, 'z'
L4:
    MOV AH, 2
    INT 21H
    DEC DL
    CMP DL, 'a'
    JAE L4

    MOV DL, 'Z'
L5:
    MOV AH, 2
    INT 21H
    DEC DL
    CMP DL, 'A'
    JAE L5

    MOV DL, '9'
L6:
```

```
    MOV AH, 2
    INT 21H
    DEC DL
    CMP DL, '0'
    JAE L6

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

0.23 Input A character print E if ASCII is even or O if odd

```
.MODEL SMALL
.STACK 100H
.DATA

MSG1 DB 'Enter a character: $'

.CODE
MAIN PROC

MOV AX,@DATA
MOV DS,AX

; print message
MOV AH,9
LEA DX,MSG1
INT 21H

; input character
MOV AH,1
INT 21H

; newline
MOV AH,2
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

; check even or odd
AND AL,1
JZ EVEN

; if odd
MOV AH,2
MOV DL,'O'
INT 21H
JMP END_PROGRAM

EVEN:
MOV AH,2
MOV DL,'E'
INT 21H

END_PROGRAM:

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

0.24 Input A letter and print the next 5 and previous 5 letters

```
.MODEL SMALL
.STACK 100H
.DATA

MSG DB 'Enter a letter: $'

.CODE
MAIN PROC

MOV AX,@DATA
MOV DS,AX

; print message
MOV AH,9
LEA DX,MSG
INT 21H

; input letter
MOV AH,1
INT 21H
MOV BL,AL

; convert case
CMP BL,'a'
JL UPPERCASE

SUB BL,32
JMP CONT

UPPERCASE:
ADD BL,32

CONT:

MOV BH,BL

; newline
MOV AH,2
MOV DL,0DH
INT 21H
MOV DL,0AH
INT 21H

; -------- NEXT 5 LETTERS --------

MOV CX,5

NEXT_LOOP:
INC BL
CMP BL,'Z'+1
JNE PRINT_NEXT
MOV BL,'A'

PRINT_NEXT:
MOV DL,BL
MOV AH,2
INT 21H
LOOP NEXT_LOOP

; newline
MOV DL,0DH
MOV AH,2
INT 21H
MOV DL,0AH
INT 21H

; -------- PREVIOUS 5 LETTERS --------

MOV BL,BH
MOV CX,5
```

```
PREV_LOOP:
DEC BL
CMP BL,'A'-1
JNE PRINT_PREV
MOV BL,'Z'

PRINT_PREV:
MOV DL,BL
MOV AH,2
INT 21H
LOOP PREV_LOOP

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

0.25 Store the lenght of a string in DL register

```
.MODEL SMALL
.STACK 100H
.DATA

in_string DB 'IUT is an International University',0
    Dh,0Ah,'$'

.CODE
MAIN PROC

MOV AX,@DATA
MOV DS,AX

; display string
MOV AH,9
LEA DX,in_string
INT 21H

; initialize
LEA SI,in_string
MOV DL,0

COUNT_LOOP:

MOV AL,[SI]     ; read character
CMP AL,'$'      ; check end of string
JE END_COUNT

INC DL          ; increase counter
INC SI          ; move to next character
JMP COUNT_LOOP

END_COUNT:

MOV AH,4CH
INT 21H

MAIN ENDP
END MAIN
```

0.26 Sum of Square of N numbers

```
.model small
.stack 100h
.data

msg1 db "Enter value of N (2-9): $"
msg2 db 13,10,"The result is: $"

array db 1,2,3,4,5,6,7,8,9
RESULT dw 0

.code
main proc

mov ax,@data
mov ds,ax

; print message
mov ah,9
lea dx,msg1
int 21h

; input ASCII
mov ah,1
int 21h

sub al,30h     ; ASCII -> number
mov cl,al

mov si,0

sum_loop:

mov al,array[si] ; get number
mov ah,0
mov bl,al
mul bl           ; AX = AL * BL (square)

add RESULT,ax

inc si
dec cl
jnz sum_loop

; print text
mov ah,9
lea dx,msg2
int 21h

; print result (AX)
mov ax,RESULT
call print_num

mov ah,4ch
int 21h

main endp


print_num proc
mov bx,10
mov cx,0

divide:
mov dx,0
div bx
push dx
inc cx
cmp ax,0
jne divide

print:
pop dx
add dl,30h
mov ah,2
int 21h
```

```
loop print

ret
print_num endp

end main
```

0.27 Find GCd AND LCM of 2 numbers

```
.model small
.stack 100h
.data

msg1 db "Enter two digits: $"
msg2 db 13,10,"GCD = $"
msg3 db 13,10,"LCM = $"

num1 db ?
num2 db ?

GCD dw ?
LCM dw ?

.code
main proc

mov ax,@data
mov ds,ax

; ask input
mov ah,9
lea dx,msg1
int 21h

; first digit
mov ah,1
int 21h
sub al,30h
mov num1,al

; space input (optional)
mov ah,1
int 21h

; second digit
mov ah,1
int 21h
sub al,30h
mov num2,al

; call GCD procedure
call find_gcd

; call LCM procedure
call find_lcm

; print GCD
mov ah,9
lea dx,msg2
int 21h

mov ax,GCD
add ax,30h
mov dl,al
mov ah,2
int 21h

; print LCM
mov ah,9
lea dx,msg3
int 21h

mov ax,LCM
add ax,30h
mov dl,al
mov ah,2
int 21h

mov ah,4ch
int 21h

main endp


; -------- GCD PROCEDURE --------
find_gcd proc

mov al,num1
mov bl,num2

gcd_loop:
cmp al,bl
je gcd_done
ja a_big
sub bl,al
jmp gcd_loop

a_big:
sub al,bl
jmp gcd_loop

gcd_done:
mov ah,0
mov GCD,ax
ret

find_gcd endp


; -------- LCM PROCEDURE --------
find_lcm proc

mov al,num1
mov bl,num2
mul bl       ; AX = num1 * num2

mov bx,GCD
div bx       ; AX = (num1*num2)/GCD

mov LCM,ax
ret

find_lcm endp

end main
```

0.28 Search if an digit is in an array or not using macro

```
macarray MACRO DIGIT
    MOV BL,DIGIT
    LEA SI,ARRAY
    MOV CX,SIZE
    CALL SEARCHPROC
ENDM
```

```
org 100h

.DATA
ARRAY DB 2,0,4,7,1,9
SIZE EQU 6
MSG1 DB 0DH,0AH,"Enter Digit: $"
MSG2 DB 0DH,0AH,"Digit Found$"
MSG3 DB 0DH,0AH,"Digit Not Found$"

KEY DB ?

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV DX,OFFSET MSG1
    MOV AH,09H
    INT 21H

    MOV AH,01H
    INT 21H
    SUB AL,30H
    MOV KEY,AL

    macarray KEY

    MOV AH,4CH
    INT 21H

MAIN ENDP

SEARCHPROC PROC
    NEXT:
        CMP BL, [SI]
        JE FOUND
        INC SI
        LOOP NEXT

    MOV DX, OFFSET MSG3
    MOV AH, 09H
    INT 21H
    RET

    FOUND:
        MOV DX,OFFSET MSG2
        MOV AH,09H
        INT 21H
        RET

SEARCHPROC ENDP

END MAIN
```